

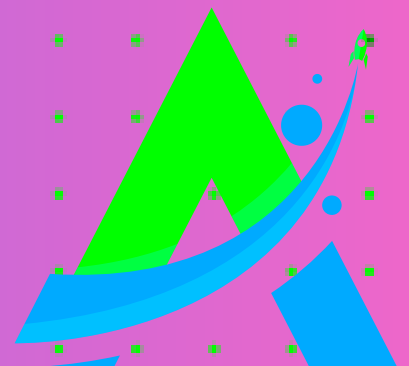
Programación *reactiva* con

ANGULAR

SIGNALS

Variables
que **notifican** sus
cambios

@albertobasalo



Contenedores reactivos que **envuelven un valor y notifican** los cambios a los interesados.

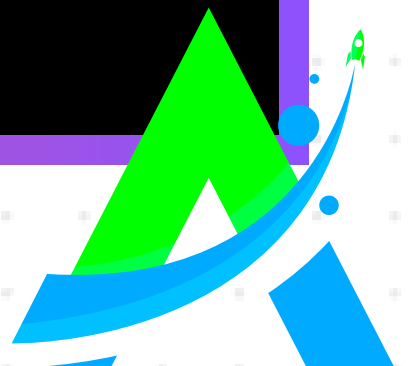
Sintácticamente se crean con **funciones** que retornan tipos, que a su vez son invocables como funciones.

1. Declaración: función constructora

```
user = signal({  
  name: 'Peter',  
  theme: 'Dark',  
  credits: 100  
});
```

2. Consumo: función evaluadora

```
<article>  
  <h2>{{ user().name }}</h2>  
  <p> Credits: {{ user().credits }}  
</article>
```



Los cambios se realizan invocando **métodos** expuestos por el tipo devuelto por la función constructora **WritableSignal**.

3. Cambio de estado

```
user.set({  
  name: 'Clark',  
  theme: 'Light',  
  credits: 100  
})
```

4. Actualización

```
user.update((current) => {  
  return {  
    ...current,           // It's a clone!  
    credits++             // Mutated!  
  };  
})
```



Los cambios pueden **derivar** en más cambios... o en la ejecución de procedimientos.

En ambos casos se usan **funciones de segundo orden** como argumentos.

Angular invoca esas funciones cuando un **disparador** lo requiere.

5. Computaciones

```
userCredits = computed(() => {  
  const credits = user().credits;  
  if(credits<=0) return 'NO CREDITS';  
  return `${credits} credits`;  
});  
console.log(userCredits()); // a string
```

6. Efectos

```
theme = computed(() => user().theme )  
changeTheme = effect(() => {  
  const myTheme = theme(); // Trigger  
  dom.setAttribute(  
    "data-theme", myTheme); // Action  
})
```



Los cambios se **propagan** a componentes *hijos* sustituyendo al clásico decorador **@Input()**.

Pueden estar *tipadas* y ser marcadas como requeridas.

Los parámetros del **router** también se pueden recibir como señales. Adios al *ActivatedRoute*

7. Input desde el padre

```
public user = input<User | undefined>();  
public user = input<User>(defaultUser);  
public user = input.required<User>();  
  
// At parent.component.html  
<child [user]= "anyUserProp" />
```

8. Input desde el *router*

```
// At app.routes.ts  
const routes = [  
  {  
    path: '/users/:id',  
    component: UserComponent  
  }  
]  
  
// At user.component.ts  
public id = input<string>();
```

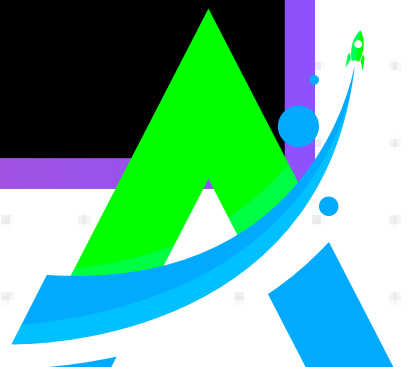


Las señales de entrada son *read-only*. Pero, también se pueden cambiar y **notificar** esos cambios.
Esas señales se convierten en **modelos**.

9. Modelos actualizables

```
// The model at the child
// Similar declaration as any input
public user = model.required<User>();
// but with changes
user.update((current) => {
  return {
    ...current,
    credits = 666
  };
});

// Back at the parent
// The good old [(banana-in-a-box)]
<child [(user)]="myUser"/>
// Have always the latest value
console.log(myUser().credits); // 666
```



Si sólo se pretende **notificar** cambios, hay una sintaxis *similar* a la de las señales.

En sustitución del clásico **@Output**

10. Salidas (similares a las señales)

```
// declaration similar to input
public buyCredits = output<number>();

// just like old event emitters
buyCredits.emit(666)

// Back at the parent... is an event
<child (buyCredits)="onBuy($event)"/>

onBuy(credits: number){
  console.log(credits);
}
```



El tipo más avanzado de señal es la que combina la capacidad de actualización **derivada y procedimental**.
Es una **señal computada que permite cambios** y recálculos.
Gran poder y gran responsabilidad.

11. Señales enlazadas

```
themes = signal([`Light`, 'Dark']);  
  
// declared like a computed...  
myTheme = linkedSignal(  
  () => themes()[0];    // Initialized  
);  
console.log(myTheme()); // Light  
  
// but mutable  
myTheme.set(themes()[1]);  
console.log(myTheme()); // Dark  
  
// and computed again!  
themes.set([`Blue`, 'Red']);  
console.log(myTheme()); // Blue
```



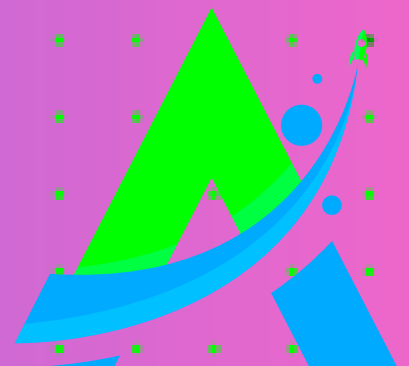
*Las **señales** necesitan recursos para enterarse de actualizaciones **asíncronas**...*

ASYNC

RESOURCE

Interoperabilidad con
productores asíncronos
como **HttpClient**, fetch...

@albertobasalo



Lo que ocurra en el **futuro** nadie lo sabe.

Hay que tener **recursos** para responder a cualquier eventualidad.

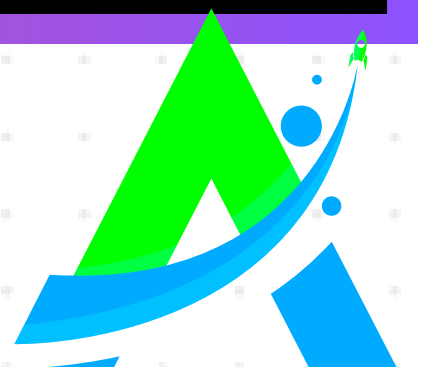
Este **tipo** dispone de señales para notificar sucesos y de métodos para provocarlos.

12. El tipo Resource

```
const userResource: ResourceRef<User>;

// Property signals
userResource.hasValue(); // Boolean
userResource.isLoading(); // Boolean
userResource.value(); // User or throw
userResource.error(); // Error or undefined
// idle error loading reloading resolved local
userResource.status();

// Mutating methods
userResource.set(newUser);
userResource.update((s)=> (...s,mutation));
userResource.reload();
userResource.destroy();
```



Dispones de tres **fuentes generadoras** de recursos con funciones reactivas que actúan como **disparadores** de procesos asíncronos. Usa un constructor u otro según la naturaleza del **productor**.

13. Promesas: resource()

```
const userResource: ResourceRef<User> =  
  resource({  
    params: () => url + userId(),  
    loader: (options) =>  
      fetch(options.params)  
        .then((res) => res.json())  
  })
```

14. Streams: rxResource()

```
const userResource: ResourceRef<User> =  
  resource({  
    params: () => userId(),  
    stream: (options) =>  
      of(users[options.params])  
  })
```



Por último, el viejo y confiable **HttpClient** que se integra en su propia función con dos sobrecargas...

La más sencilla, y recomendable, es una **función reactiva** que lo dispara.

15. HttpClient: httpResource()

```
// Using a reactive function to build the url  
const userResource: ResourceRef<User> =  
    httpResource(() => url + userId())
```

```
// Use an object to control more the request  
const userResource: ResourceRef<User> =  
    httpResource({  
        url: `http://your-server.api/${userId()}`,  
        method: 'GET',  
        headers: [], params:{}, body:{}  
    })
```

Aunque admite POST, PUT...

no está recomendado para enviar cambios al API .

Pero lo podemos simular fácilmente con un poquito de **RxJS...**



Para realizar **cambios** en el servidor (mutaciones con POST,PUT...) crea servicios que implementen parcialmente la interface **Resource<>**

16. HttpClient: Resource<>

```
class UserLogin implements Resource<User>{
    value = signal<User>();
    error = signal<string|undefined>();
    status = signal<ResourceStatus>('idle');

    http = inject(HttpClient);

    send(login: LoginDTO): void {
        this.error.set(undefined);
        this.status.set('loading');
        this.http.post('url', login).subscribe({
            next: (res) => {
                this.value.set(res);
                this.status.set('resolved');
            },
            error: (err) => {
                this.error.set(err);
                this.status.set('error');
            }
        })
    }
}
```



*Y esto **aún** no son todos los usos que te ofrece*

ANGULAR

SIGNALS

Acceso a elementos de las
templates, stores reactivos...,
y muy pronto formularios!!

Sigue atento a mis señales

@albertobasalo

